# [Khanh Pham] - Linux OS Overview & Kernel Module/Driver Basics

# Linux OS Overview & Kernel Module/Driver Basics

> In the previous parts, we understand about process, thread, IPC,... Most of them belong to a side of linux: **User space**. So in this document, I will write about another side: **kehrnel space**

First of all, before going to kernel concept, we need understand about overview of linux:

# Overview of linux

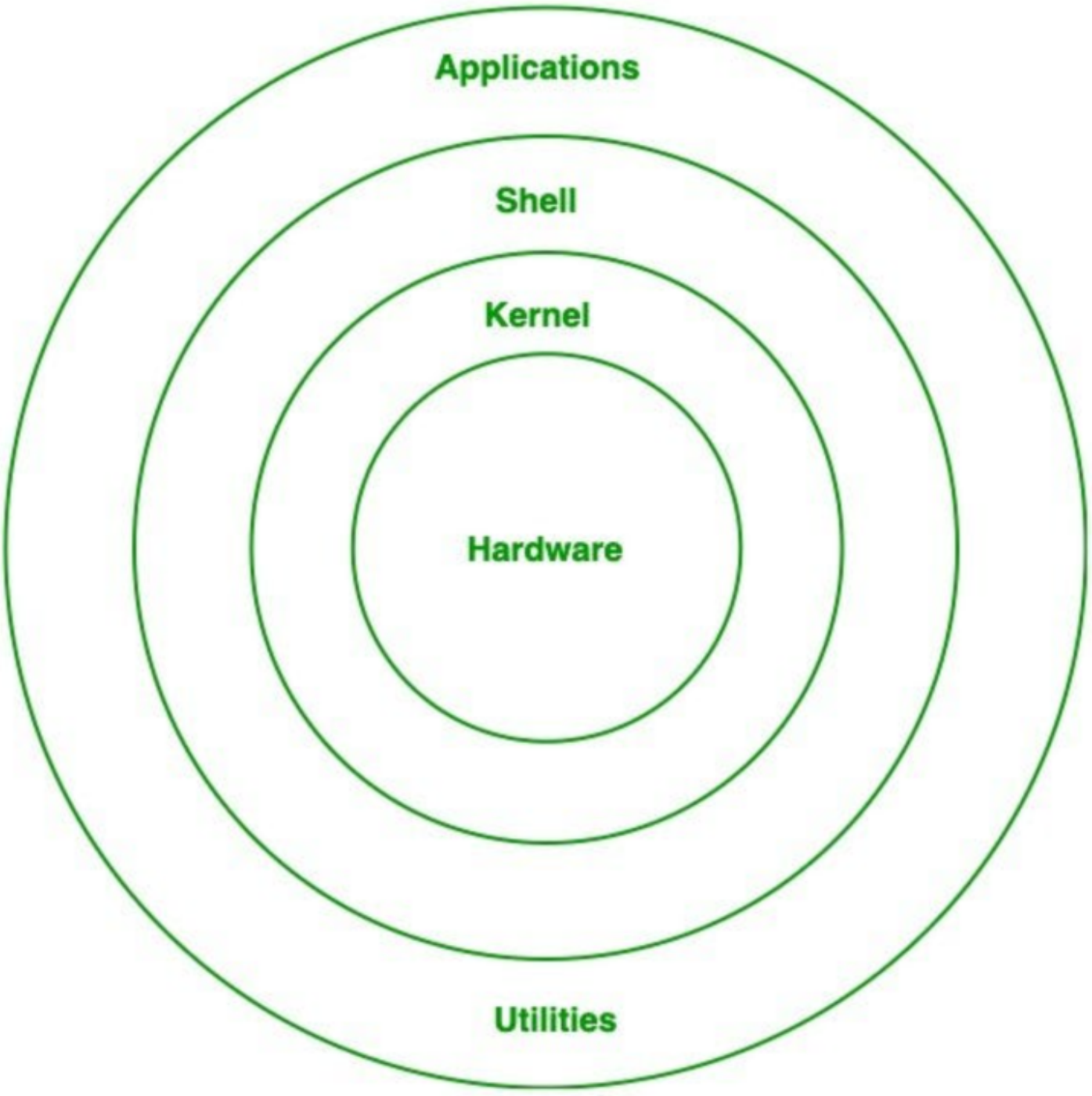
Operation system(OS) is application that manage both hardware and software of a device. It helps users can use and manage hardware without caring about the implement bellow.

Linux is open-source OS that created by Linus Torvalds in 1991. It bases on traditional UNIX OS.

Linux have 4 important components:

- **Hardware**: The lowest layer of Linux architecture, it consists of hardware components like: CPU, RAM, Input-Output device
- **Kernel**: The core of linux OS, it helps higher layers can communicate with lower layer(hardware).

- **Shell**: Shell is the interface that interacts with humans and processes the commands that are given for the execution.
- **Application and utilities**: Applications are the programs that the user runs on top of the architecture. It can interact with lower layers by using the command of system utilities like `ls`, `top`, `rm`, ...




Now we go to the most important layer in linux: **Kernel component**

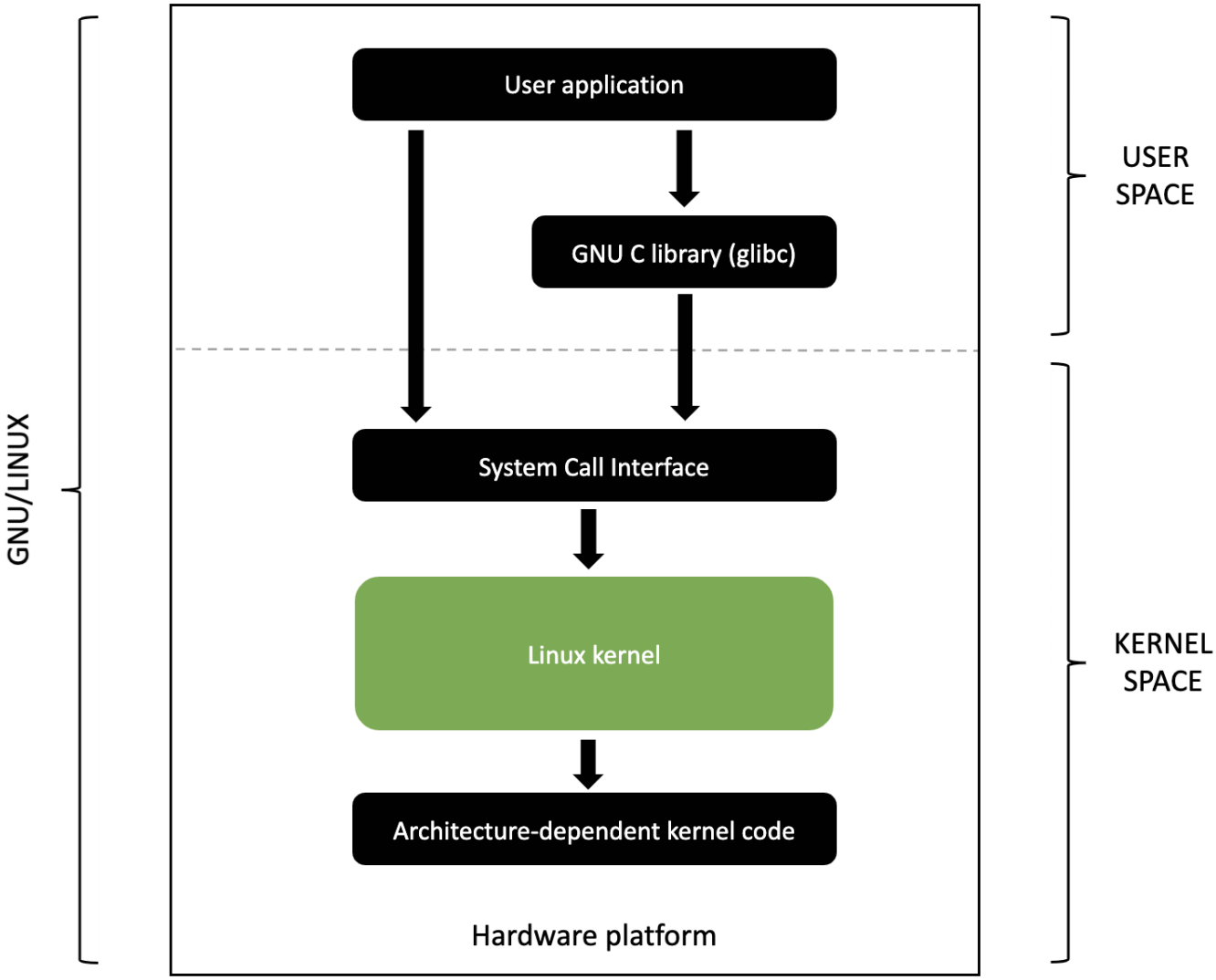
# Kernel

Before writing about the deeper knowledge of kernel, we must understand the different between user and kernel:

# Compare user-space and kernel-space

User-space is the area that belong to users. That mean we can write every program that help us every thing that we need. However, when we need interact with system component like: RAM, File, ICP object, we must request to kernel space by using system call. The reason why we need both user and kernel instead of calling directly to hardware:

- **Security**: apps you run can't access anything they should not be accessing
- **Stability**: isolating apps allows Kernel to track resources and safely terminate and cleanup misbehaving programs
- **Compatibility**: Kernel provides stable API for apps (e.g. CreateFile function), and can change anything under it without apps noticing



> When the code of kernel space has unrestricted access to all CPU instructions, hardware devices, and system memory. Therefore, Not like process in kernel space, we can make have exception, other processes will not crash. But in kernel space, the **device** can get kernel panic error.

As we know from shared memory document, in 64-bit CPU has 48 bits for virtual address. (128 TB for kernel and 128 for user). However, not like user space, each process have seperate virual memory. In kernel, all process share the kernel virtual memory.

When process in user mode try to access to virtual address of kernel, the kernel will send `SIGSEEV` signal to this process.

Now after understand about the reason why we need kernel, we will go to sections that describe the ability of kernel:

# Kernel overview

There are 5 crucial subsystems in linux kernel:

- Process management
- Memory management
- File system
- Network
- Device driver

## Process management

I don't write again about the process topics like memory layout, IPCs or socket here. In this section, I will focus on process life cycle and scheduled of process.

If in user-space, we can watch every processes by memory layout, but in kernel-space, kernel know process by process control block(PCB)

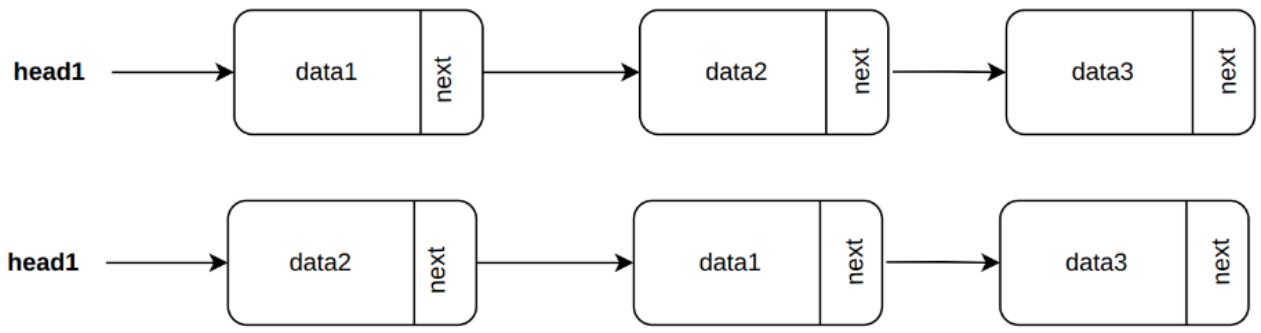
> PCB is represented by task_struct in kernel, it saves every information that help kernel can manage process like PID, state, CPU register, ...

- Kernel use some data structures to store pcb. Linklist to list all pcb, tree to know the parent-child relationship, hash-table to find pcb with specific PID in O(1)

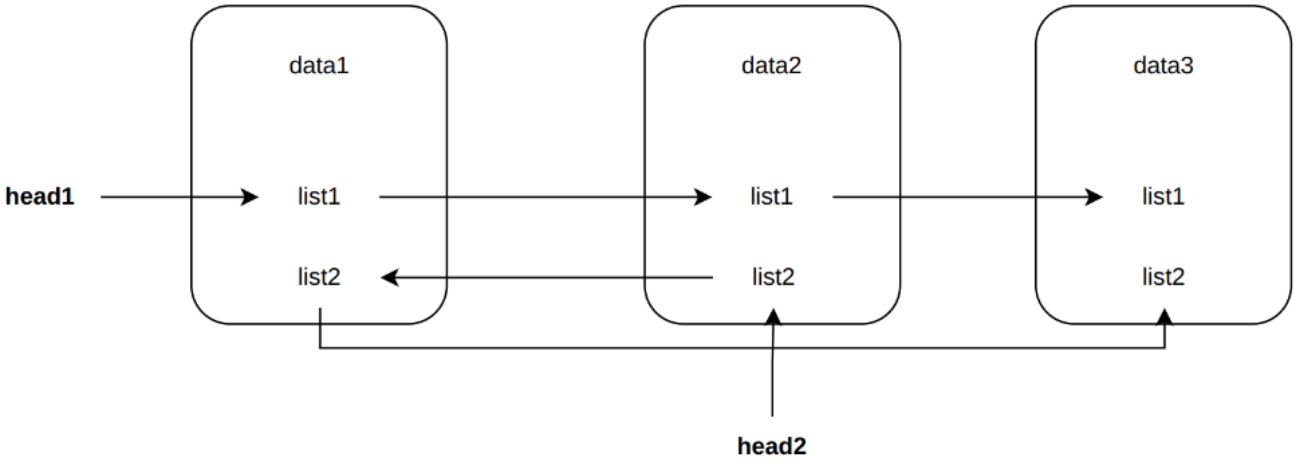
>

> Kernel don't use the copy of PCB of each process for storing data structure, it use the **Intrusive Linked List** inside:

Not like normal linked list, each node stored both data and the pointer to next node. The technique above use the struct element as a pointer to another element pointer of other PCBs.




This method will waste the kernel resource.

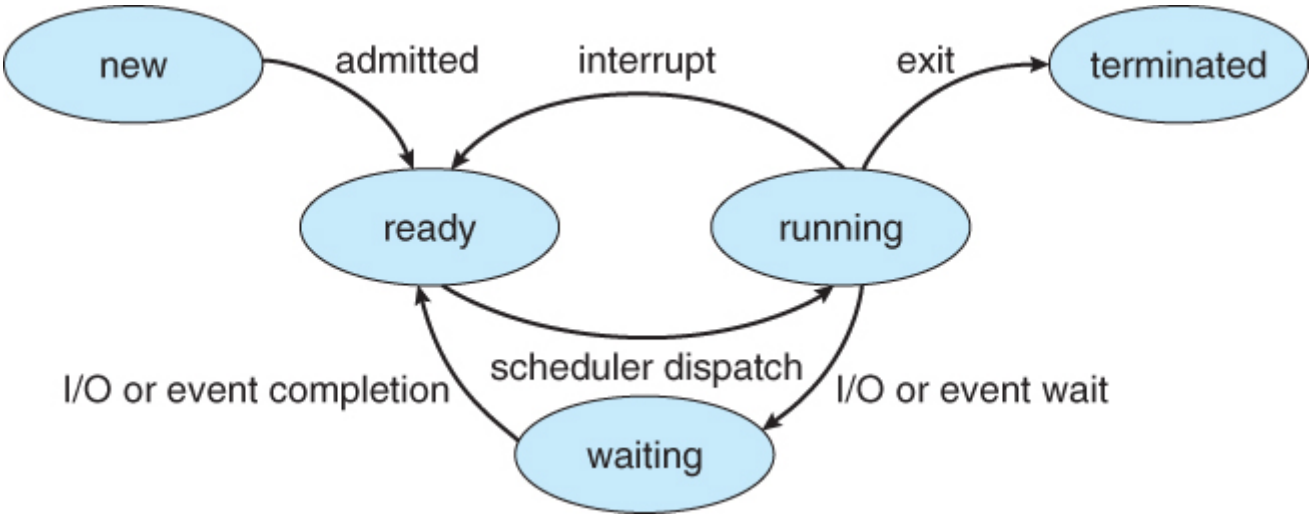



The pros of Intrusive Linked List:

- Create a new node of every data structure faster, just use the linklist element inside struct instead of creating new data node
- A node can belong to more than 1 data structure without using a copy of node
- Improve cache peformance, faster read data( don't need to do cache miss when read new node of list)

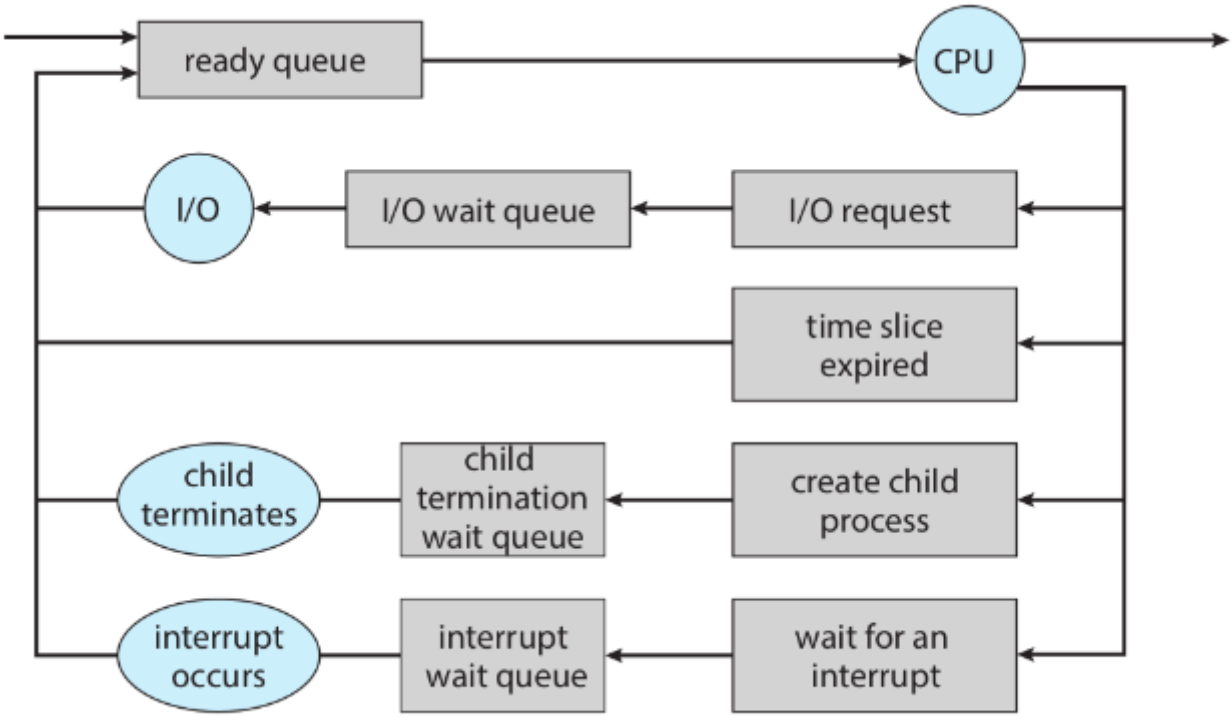
> Because the next node is the element of list, we need a function to point to the head struct: `container_of` to move from address of node element to the first address of `task_struct`

Every processes have 5 states like the image bellow:




- The new state is when process is creating. The kernel is creating PCB, prepare resource for process
- The ready state is when process is created and waiting for scheduling
- The running state is when process is chose by scheduler and running.
- The terminated state is when process done and terminate
- The waiting state is when process need other resource like I/O, timer, other events

> Every events in kernel have specific queues: We can see the example bellow:

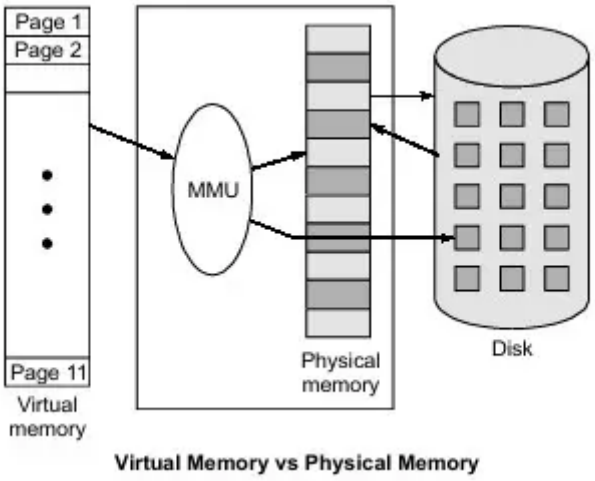



# Memory management

The kernel of modern linux use page and virtual address to manage memory.

In every device, they have RAM. Each RAM have lots of memory address(physics memory). Howver, we use virtual address instead of using physics address in RAM. There are some reasons:

- **Process isolation**: One process can't see the another process memory
- **Simplifying application writing**: Each process thinks it has a contiguous set off memory addresses, and the same set each time
- **Simplifying for creating process**: We don't need to care about the physics RAM in user devices. Just use all virtual address




**Virtual Memory vs Physical Memory**

> MMU is the element translate from virtual address to physics address

However, mmu don't manage each seperate mapping address, it use page address. Normally, each page have 4 KB address.

- In virtual address, we call page
- In physics address, we call page frame
- We call the mapping of page to page frame is page table

> When we access to address without belonging to any page, the MMU will make page fault, then kernel will check the address in VMA, if this address isn't belong to any VMA, the process will receive `SIGSEEV` and crash

For example:

```
int main()
{
    char *a = malloc(sizeof(char));
    *a = 5;
    printf("%d\n", *a);
    getchar();
    a += 131 * 1024;
    printf("%d\n", *a);
}
```

When we run this process and run `cat /proc/pid/maps` of this process:

```
61314b112000-61314b113000 rw-p 00003000 103:08 2376349
/home/khanhnp/C_programming/test
613186acd000-613186aee000 rw-p 00000000 00:00 0                          [heap]
7ff2e8a00000-7ff2e8a28000 r--p 00000000 103:07 1714624                   /usr/lib/x86_64-
linux-gnu/libc.so.6
```

We can see OS allocate for this programm 132 * 1024 bytes page of heap. If we access to the address 131 * 1024( still belong to page), there will be nothing error. But if we access to 132 * 1024 addess( out of page), the process will received `SIGSEEV` signal and crash

```
5 // 131 * 1024


0
```

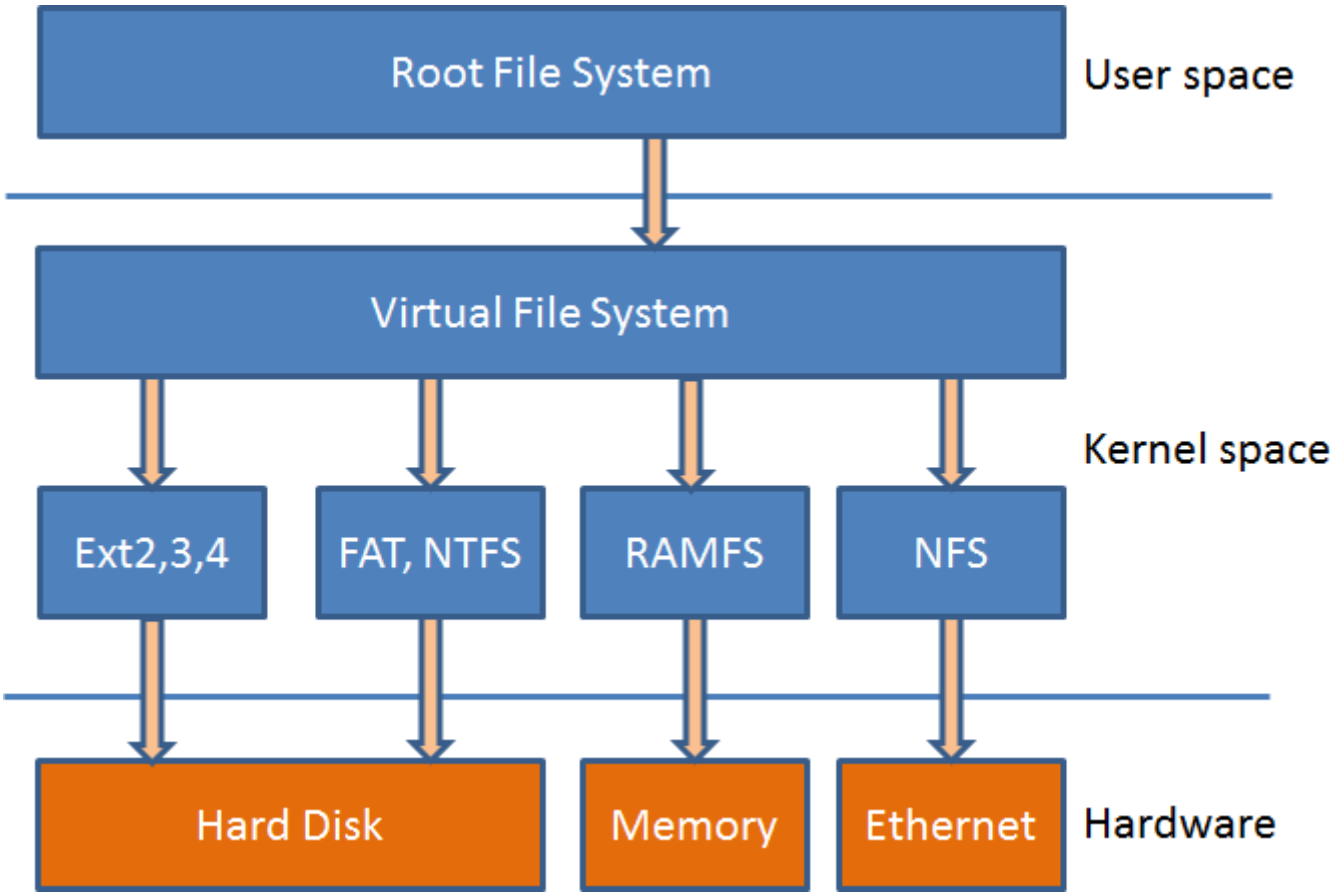
```
5 // 132 * 1024


Segmentation fault (core dumped)
```

## File system

In memory, all files stored by only 0 and 1. To present them, there are many way: EXT4, FAT32, NTFS,... However, when we use file system call like read, write, open, we don't need to care about the type of file system. Kernel will do this by using Virtual file system(VFS)

> VFS is the abtraction layer in kernel space. The idea of VFS is using function pointer. When process use system call to file, kernel will receive the file descritor from process. Then kernel will check the file system type of this file. After that, kernel will choose exaclty the suitable function for this file.




In VFS, there are 4 main components:

- Superblock: A superblock object represents a mounted filesystem( `/` or `/home` ).

- Inode: This object represents files in system. A file can have more than 1 file descriptor, but kernel always watch this file by only 1 struct inode. This struct consist of the `inode_operations`, this include all suitable function of this file type.
- Dentry: This object represents the dictionary in system. Each element is 1 struct object( `/sys/dev` become 2 object: `sys` and `dev`)
- File: Each time when process call `open`, kernel will create a struct file to represent this file. This struct also have `file_operations` to directly interact with this file

Let's take an example of opening file `/var/log/syslog`

- The first step use dentry to check the dentry of file `syslog`
- From the dentry of `syslog`, VFS get the inode of this file
- Then create a file object with `file_operations` same `inode_operations` from inode struct
- After that, kernel will return a `fd` which point to this file struct. Therefore, when process need other interaction with this filesystem, VFS just use functions in `file_operations`

# Network

In network subsystem, I will write about the process of a packet from network card to user process.

1. Sending(TX - transmit path):

- Application call `send` to send a data payload
- Socket layer: Moving data from user space to kernel space, pack data become paket object(sk_buff)
- Transport layer: Add TCP/UDP header
- Network layer: Add Ip header and look for the routing table to know suitable network card
- Driver: Push packet to network card to send

2. Receiving(RX - receive path)

- Network card: Receive from hardware, push to RAM by DMA and alert to kernel
- Network layer: Unpack IP header, check the IP address and determine receive or forward
- Transport layer: Unpack TCP/UDP header, check destination port and look for suitable socket
- Socket layer: Give data payload to receive buffer of socket in kernel
- Application: call `recv` and get data from kernel to user space

# Device driver

Device driver is the lowest component in kernel. It help kernel can communicate with real hardware. The purpose of device driver:

- **Hardware abtraction**: It provide for kernel a standard API, so kernel don't need care about type of device.
- **Command translation**: Translating from higher command to lower request in hardware

# Linux kernel module

We can load a custom dynamic module to kernel. This action will update kernel without building and loading again all kernel.

The pros of loadable module with built-in module

| | Built-in | Loadable |
|---|---|---|
| Stored position | Inside original kernel | Inside kernel module object(.ko) |
| Build and load | When device power on | Everytime when device is running |
| Ram consuming | Every device driver on RAM | Loaded devive driver on RAM |
| Update kernel | Need to build and load all kernel again | Just load new module to kernel |

# Kernel module example turn on/off led

- Device: Raspberry Pi 4
- Idea: Using registers information in https://pip-assets.raspberrypi.com/categories/545-raspberry-pi-4-model-b/documents/RP-008248-DS-1-bcm2711-peripherals.pdf and data from device file to manage the value of GPIO pin in device.

Source:

```
#include <linux/kernel.h>
#include <linux/init.h>
#include <linux/module.h>
#include <linux/proc_fs.h>
#include <linux/slab.h>
#include <linux/uaccess.h>
#include <asm/io.h>

#define BCM2711_GPIO_ADDRESS 0xfe200000
#define PROC_FILENAME "khanh"
#define LLL_MAX_USER_SIZE 1024
static unsigned int pin = 17;
static char data_buffer[LLL_MAX_USER_SIZE + 1] = {0};

static unsigned int *gpio_registers = NULL;
```

```
static void gpio_pin_on(void) {
	unsigned int fsel_index = pin/10;
	unsigned int fsel_bitpos = pin%10;
	unsigned int* gpio_fsel = gpio_registers + fsel_index;
	unsigned int* gpio_on_register = (unsigned int*)((char*)gpio_registers + 0x1c);

	*gpio_fsel &= ~(7 << (fsel_bitpos*3));
	*gpio_fsel |= (1 << (fsel_bitpos*3));
	*gpio_on_register |= (1 << pin);

	return;
}



static void gpio_pin_off(void) {
	unsigned int *gpio_off_register = (unsigned int*)((char*)gpio_registers + 0x28);
	*gpio_off_register |= (1<<pin);
	return;
}

static ssize_t driver_proc_write(struct file *file_pointer, const char __user *user_buffer,
size_t count, loff_t *offset) {

	unsigned int value = UINT_MAX;

	memset(data_buffer, 0x0, sizeof(data_buffer));

	if (count > LLL_MAX_USER_SIZE)
	{
		count = LLL_MAX_USER_SIZE;
	}

	if (copy_from_user(data_buffer, user_buffer, count))
		return 0;

	if (sscanf(data_buffer, "%d", &value) != 1)
	{
		printk(KERN_ALERT "INVALID DATA FORMAT\n");
		return count;
```

```
    }

    if (value != 0 && value != 1)
    {
        printk(KERN_ALERT "INVALID ON/OFF VALUE\n");
        return count;
    }

    if (value == 1)
    {
        gpio_pin_on();
    } else if (value == 0)
    {
        gpio_pin_off();
    }

    return count;
}

static struct proc_dir_entry *proc_file;

struct proc_ops proc_fsops = {
    .proc_write = driver_proc_write
};

static int __init my_init(void) {
    printk(KERN_INFO "[INIT_GPIO_PIN] EXECUTION STARTED\n");

    gpio_registers = (int*)ioremap(BCM2711_GPIO_ADDRESS, PAGE_SIZE);

    if (gpio_registers == NULL) {
        printk(KERN_ALERT "FAILED TO MAP GPIO MEMORY TO DRIVER\n");
        return ENOMEM;
    }

    printk(KERN_INFO "SUCCESSFULLY MAPPED GPIO MEMORY TO THE DRIVER\n");

    proc_file = proc_create(PROC_FILENAME, 0666, NULL, &proc_fsops);

    if (proc_file == NULL) {
```

```
    printk(KERN_ALERT "PROC ALLOCATION FAILED\n");
    return -1; 
}

 printk(KERN_INFO "PROC ALLOCATION SUCCESSFUL\n");

 return 0;
}

static void __exit my_exit(void) {
 printk(KERN_INFO "[EXIT_GPIO_PIN]: EXECUTION STARTED\n");

 iounmap(gpio_registers);
 proc_remove(proc_file);

 printk(KERN_INFO"[EXIT_GPIO_PIN]: EXECUTION DONE\n");
}

module_init(my_init);
module_exit(my_exit);

MODULE_LICENSE("GPL");
MODULE_AUTHOR("Khanh");
MODULE_DESCRIPTION("On/Off led on pi 4");
```

**Explain**:

- In RP4, the base physics address of GPIO is `0xfe200000`, we use `ioremap` function to map this address to kernel virtual address.
- Creating a device file in `/proc/` with the behavior of writing file is file_op function: `driver_proc_write`

After that, the result is:

```
khanhnp@raspberrypi:~ $ sudo insmod khanh.ko // insert module to kernel

khanhnp@raspberrypi:~ $ echo 1 > /proc/khanh // turn on the led

khanhnp@raspberrypi:~ $ echo 0 > /proc/khanh // turn off the led
```

---

Revision #4
Created 2026-10-12 01:39:51 UTC by Khanh Pham
Updated 2026-10-14 10:35:53 UTC by Khanh Pham